PROJECT REPORT

Quality Assurance AI

AI-Powered Textile & Fabric Defect Detection System

Submitted BY: Ayush Dwivedi
Reg. No.: 12513935
Project Guide: Mr. Sreesh Bhat
Date: 20th Jul 2026

Computer-Vision-Based Automated Fabric Inspection

Built with YOLOv8, Flask, OpenCV & Bootstrap 5

Table of Contents

1. Executive Summary
2. Introduction & Industry Background
3. Problem Statement & Objectives
4. Related Approaches to Textile Quality Inspection
5. System Architecture
6. Technology Stack
7. Dataset
8. Model Training Methodology
9. Model Performance Evaluation
10. Application Features
11. User Interface & Experience Design
12. Sample Detection Results
13. Backend API Summary
14. Project Structure
15. Testing & Validation
16. Deployment Considerations
17. Limitations & Future Work
18. Conclusion

Appendix A: Dependencies

Appendix B: Class Index Mapping

1. Executive Summary

Quality Assurance AI (QA.AI CORE) is a web-based, AI-powered visual inspection system built to automate defect detection in textile and fabric manufacturing. The application uses a custom-trained YOLOv8 object detection model to identify common fabric defects — holes and yarn defects — from uploaded images or a live camera feed, and returns annotated results with bounding boxes, confidence scores, and inference statistics in real time.

The system is delivered as a Flask web application with a dark-themed, sci-fi-styled dashboard interface, offering both single-image upload analysis and a live-camera inspection mode suitable for demonstrating real-time production-line monitoring. This report documents the motivation, architecture, dataset, training methodology, performance, features, and deployment considerations of the project in detail.

At a glance, the trained model achieves 81.3% precision and a mAP@0.50 of 75.0% across two defect classes, and the full-stack application supports both offline image analysis and real-time camera-based inspection, making it a viable proof-of-concept for AI-assisted quality control on a production floor.

2. Introduction & Industry Background

Textile manufacturing is a high-volume, fast-paced industry where even small, localized defects — a hole, a broken pick, a skipped stitch, a snag — can render an entire roll of fabric unsellable or force costly downstream rework. Traditionally, fabric inspection has relied on human inspectors visually scanning fabric as it passes over a lightboard or inspection table, a method sometimes called '4-point' or '10-point' grading.

This manual process, while still widely used, has well-documented limitations: inspector fatigue reduces detection accuracy over long shifts, subjective judgment leads to inconsistent grading between inspectors and shifts, and inspection throughput is fundamentally capped by human visual processing speed. Industry studies have long suggested that manual inspection catches only a fraction of true defects, particularly for small or subtle flaws.

Advances in computer vision and deep learning — particularly single-stage object detectors such as the YOLO (You Only Look Once) family — have made real-time, camera-based automated inspection increasingly practical, even on modest hardware. This project applies that technology directly to the textile domain, aiming to demonstrate a working, end-to-end automated inspection pipeline rather than a purely theoretical model.

3. Problem Statement & Objectives

Manual quality inspection of textiles is labor-intensive, subjective, and prone to human error and fatigue, which can allow defective fabric to pass through production lines undetected. The objective of this project is to build an automated, AI-driven quality assurance tool that can:

- Detect and localize common fabric defects (holes, yarn defects) in images with bounding boxes.
- Provide a confidence score and defect count for each inspected sample.
- Support both offline image uploads and real-time live camera inspection.
- Present results through an intuitive, visually engaging web dashboard usable on a factory floor or in a QA lab.
- Serve as an extensible foundation that can later incorporate more defect classes and production-line integrations.

4. Related Approaches to Textile Quality Inspection

4.1 Manual Inspection

The traditional baseline: human inspectors visually examine fabric under standardized lighting, marking and scoring defects according to industry grading systems. Low upfront cost but high recurring labor cost, variable accuracy, and poor scalability.

4.2 Classical Image Processing

Earlier automated approaches used rule-based image processing techniques — edge detection, texture analysis (e.g., Gabor filters, GLCM statistics), and thresholding — to flag irregularities in a fabric's periodic weave pattern. These methods can be fast but are brittle: they require careful hand-tuning per fabric type and struggle to generalize across different weave patterns, colors, and lighting conditions.

4.3 Deep-Learning-Based Detection (this project's approach)

Modern object detection networks learn defect-relevant visual features directly from labeled examples, generalizing better across varied fabric types and imaging conditions than hand-crafted rules. Single-stage detectors like the YOLO family are especially well suited to this problem because they offer real-time inference speed, a requirement for any system intended to run inline on a moving production line or live camera feed. This project adopts YOLOv8, the architecture used to train the model shipped with this application.

5. System Architecture

The application follows a simple client-server architecture. The Flask backend loads a pretrained YOLOv8 model at startup and exposes REST endpoints for image upload/inference and an MJPEG streaming endpoint for the live camera feed. The frontend is a responsive Bootstrap 5 single-page-style dashboard that communicates with the backend via JavaScript fetch calls and renders annotated results dynamically.



Figure 1 — High-level data flow through the application.

5.1 High-Level Flow

- User uploads an image (or starts the live camera) via the browser interface.
- Flask backend receives the image, saves it, and passes it to the YOLOv8 model for inference.
- The model returns bounding boxes, class labels (hole / yarn defect), and confidence scores.
- OpenCV renders the annotated image, which is saved to the results folder and served back to the browser.
- The dashboard displays the annotated image alongside inference time, object count, device used, and per-defect confidence.

5.2 Live Camera Path

For the live camera mode, the /video_feed endpoint opens the default camera device with OpenCV's VideoCapture, runs YOLOv8 inference on each captured frame, draws the resulting bounding boxes onto the frame, and yields it as part of a multipart/x-mixed-replace MJPEG stream — a lightweight approach that avoids WebSockets or external streaming infrastructure while still giving a live, continuously updating annotated feed in the browser.

6. Technology Stack

The project intentionally keeps its dependency surface small, favoring a handful of well-established libraries over a heavier framework stack:

Layer	Technology	Purpose
Backend	Python, Flask	Web server & REST API routing
AI / ML	Ultralytics YOLOv8 (YOLOv8n)	Object detection model for defect localization
Image Processing	OpenCV (opencv-python-headless)	Frame capture, annotation rendering, encoding
Frontend	HTML5, Bootstrap 5, Font Awesome, JavaScript	Dashboard UI, upload flow, live camera UI

Utilities	Werkzeug	Secure filename handling for uploads

Ultralytics YOLOv8 was chosen over training a detector from scratch because it provides a mature, well-optimized reference implementation with strong default augmentation, an efficient anchor-free detection head, and a simple Python API (model.predict(), result.plot()) that maps cleanly onto the Flask request/response cycle used throughout app.py.

7. Dataset

The model was trained on a labeled fabric-defect image dataset formatted and exported via Roboflow, evident from the '.rf.<hash>' suffix present in several of the source image filenames (e.g., defect_971-00_..._jpg.rf.992c585145bf1e2251555d7d1c49dc93.jpg). Roboflow-exported datasets are typically split into train/validation/test folders with an accompanying data.yaml describing class names and paths — referenced in the model's saved training configuration as /content/dataset/data.yaml, indicating the model was trained in a cloud notebook environment (consistent with the Tesla T4 GPU used).

7.1 Classes

- hole — punctures or torn openings in the fabric surface.
- yarn defect — irregularities in the weave such as broken picks, skipped stitches, or knots.

7.2 Observed Data Characteristics

Filenames in the uploads/results folders point to a mix of real inspection-camera captures (e.g., defect_619-00_12232-1_2022-10-07...) and supplementary stock/reference photography (e.g., istockphoto-*.jpg), suggesting the training set combines authentic production-line defect imagery with additional web-sourced fabric images to increase visual diversity. Filename tags such as 'Flip' indicate that horizontal/vertical flip augmentation was applied during dataset preparation, which helps the model generalize to defects appearing in different orientations.

7.3 Preprocessing

- Images resized/letterboxed to the model's 640×640 input resolution.
- Standard YOLOv8 augmentations during training (mosaic, flip, HSV color-jitter, scale).
- Two-class bounding-box annotation format (YOLO .txt labels) per Roboflow export conventions.

8. Model Training Methodology

Base Architecture	YOLOv8n (nano)
Task	Object Detection

Detected Classes	hole, yarn defect
Image Size	640 x 640
Epochs	30
Batch Size	16
Optimizer	Auto-selected (initial LR 0.01)
Training Hardware	NVIDIA Tesla T4 GPU
Weights File	best.pt (~6.2 MB)

YOLOv8 optimizes a composite loss made up of a box regression loss (CIoU/DFL-based), a classification loss, and a distribution focal loss (DFL) for precise bounding-box localization. The chart below shows how training and validation losses decreased over the 30 training epochs, alongside the corresponding rise in validation precision, recall, and mAP.

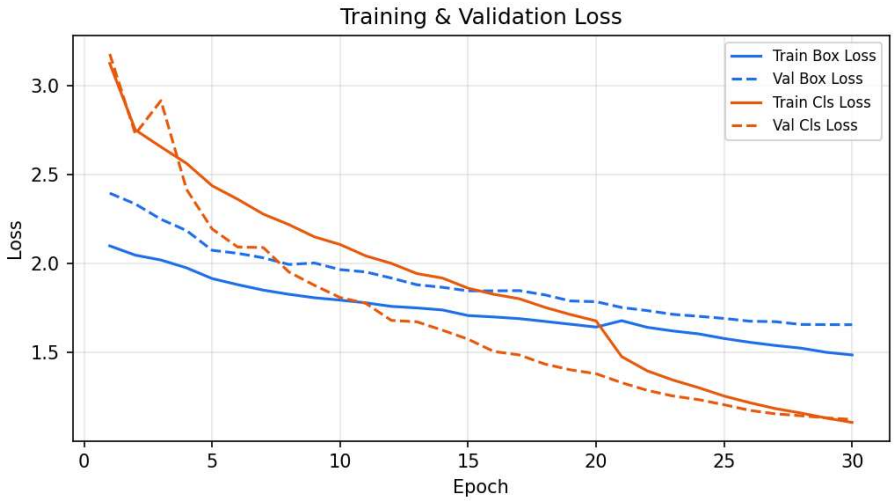


Figure 2 — Training and validation loss curves across 30 epochs.

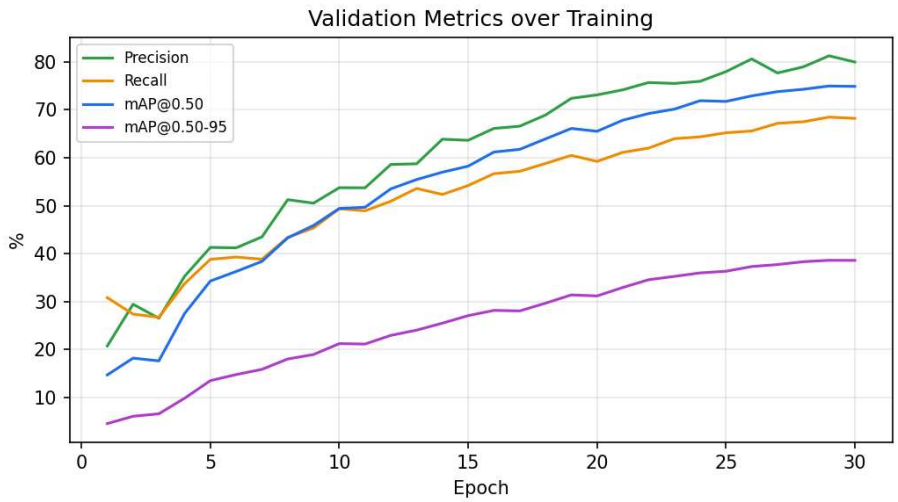


Figure 3 — Validation precision, recall, and mAP progression across 30 epochs.

The curves show rapid early gains in the first ~10 epochs followed by a more gradual improvement, which is typical of fine-tuning a compact detector on a moderately sized, domain-specific dataset. The model checkpoint saved as best.pt corresponds to the epoch with the best overall fitness score during training.

9. Model Performance Evaluation

Final validation metrics recorded in the trained checkpoint:

Metric	Value
Precision	81.27%
Recall	68.48%
mAP@0.50	74.97%
mAP@0.50–0.95	38.60%

These metrics indicate strong precision (a low false-positive rate) with moderate recall, meaning the model is conservative but reliable when it does flag a defect — a reasonable trade-off for a first-pass automated QA screening tool where flagged items can be sent for secondary human review. The lower mAP@0.50–0.95 relative to mAP@0.50 is expected: it averages performance across stricter IoU thresholds and is sensitive to bounding-box tightness, which tends to be harder to optimize for small, irregularly shaped defects like holes.

Because the dataset contains only two classes with differing visual characteristics — a well-bounded hole versus a more diffuse yarn-level irregularity — per-class performance likely varies, with compact, high-contrast holes generally easier to localize precisely than subtler yarn defects.

10. Application Features

10.1 Image Upload & Detection Mode

- Drag-and-drop or click-to-browse image upload (JPG, JPEG, PNG).
- Live image preview before running detection.
- Animated 'scanning' UI sequence while inference runs, for a production-inspection feel.
- Annotated result image with bounding boxes and defect labels, downloadable from the dashboard.
- Detection history log tracked in the sidebar for the current session.

10.2 Live Camera Mode

- Real-time video feed from a connected camera, streamed via MJPEG (multipart HTTP response).
- Per-frame YOLOv8 inference with bounding boxes drawn live on the video stream.
- Start/Stop controls and a live status indicator (STANDBY / ACTIVE).

10.3 Analysis Dashboard

- Inference time (ms), total object count, and compute device (CPU/GPU) shown per detection.
- Per-defect class name and confidence percentage breakdown.
- Circular confidence gauge and status banner (e.g., defect found / clean).
- Static system stats panel displaying model accuracy, precision, and recall from training.

11. User Interface & Experience Design

The dashboard adopts a dark 'sci-fi command console' aesthetic, intended to reinforce the sense of an automated inspection system rather than a plain data-entry form. Key design choices include:

- An animated boot sequence on first load ('Initialize SYSTEM...', 'Loading AI Model...') that frames the tool as an active AI system coming online.
- Orbitron and Rajdhani Google Fonts for a technical, futuristic typographic feel, paired with Font Awesome icons throughout.
- A 'glass card' panel style (semi-transparent, blurred backgrounds) for the navbar and content panels, giving the UI visual depth.
- A scanning-laser overlay animation with corner targeting brackets shown while an uploaded image is being processed, reinforcing the sense of active analysis.
- Color-coded neon accents (blue/cyan) for active/system elements and red/warning tones for alerts, aiding quick visual status recognition.

This styling choice is primarily oriented toward demo and presentation contexts — trade shows, stakeholder demonstrations, portfolio review — where a visually distinctive interface helps communicate the 'AI-powered' nature of the tool at a glance, while the underlying functional flow (upload → analyze → review results) remains straightforward for a non-technical operator.

12. Sample Detection Results

The following samples illustrate the model's output on real fabric images, showing bounding boxes drawn around detected defects with class labels and confidence scores.



Figure 4 — Detected hole defect with bounding box and confidence score.

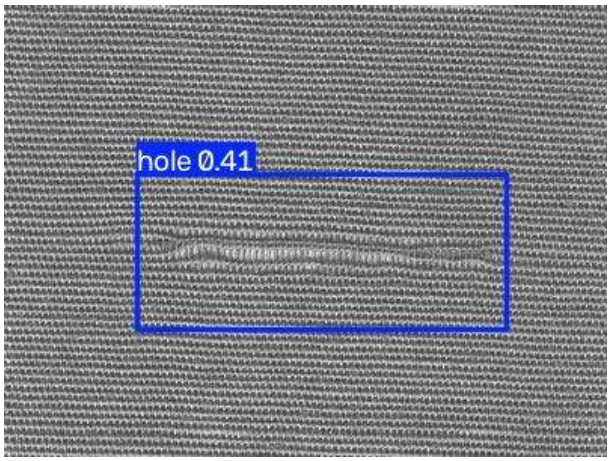


Figure 5 — Detected yarn defect (broken pick) on woven fabric sample.

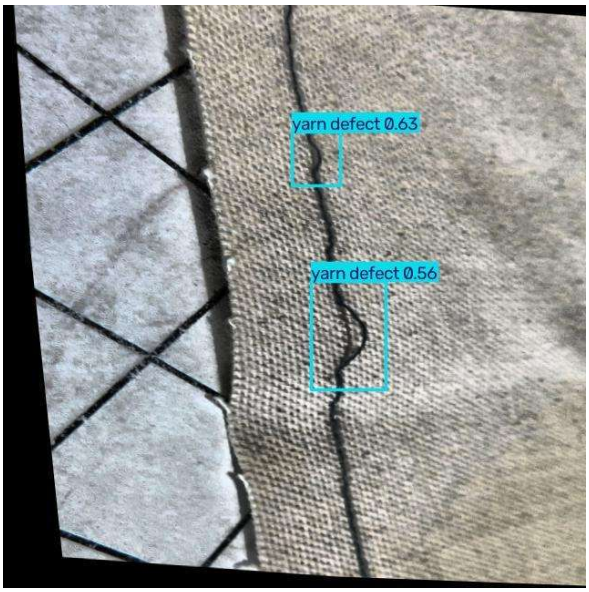


Figure 6 — Detected defect on a skipped-stitches fabric sample.

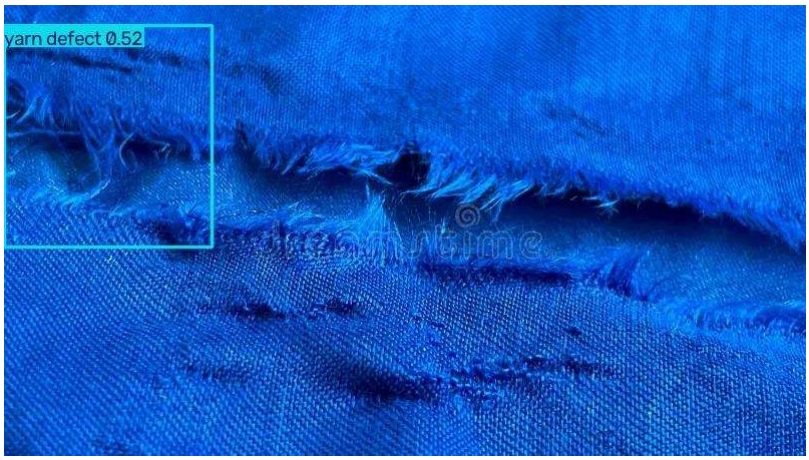


Figure 7 — Detected hole/tear on a macro fabric sample, illustrating detection at close range.

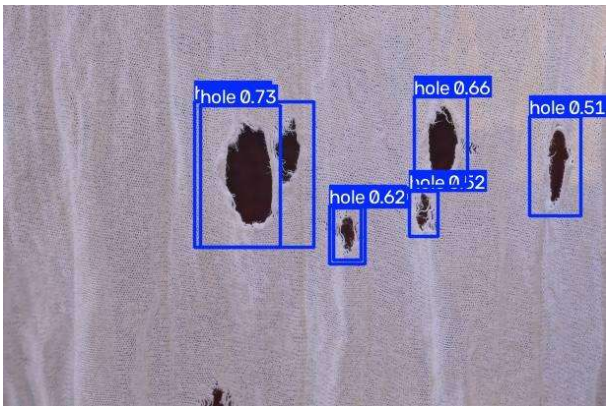


Figure 8 — Inspection sample from the results log used to validate detection consistency.

13. Backend API Summary

Route	Method	Description
/	GET	Renders the main image-upload dashboard.
/camera	GET	Renders the live camera inspection page.
/upload	POST	Accepts an image file, runs YOLOv8 inference, returns annotated image URL, detected objects, confidence, timing, and device info as JSON.
/results/<file>	GET	Serves a saved annotated result image.
/video_feed	GET	Streams live camera frames with real-time YOLOv8 annotations (MJPEG).

14. Project Structure

The codebase is organized as a standard Flask project:

- app.py — Flask application, model loading, inference & streaming routes.
- best.pt — trained YOLOv8 model weights.
- requirements.txt — Python dependencies (flask, ultralytics, opencv-python-headless, werkzeug).
- templates/ — index.html (upload dashboard) and camera.html (live feed page).
- static/css/style.css, static/js/script.js — dashboard styling and client-side logic.
- uploads/ — stores user-submitted source images.
- results/ — stores annotated output images returned to the user.

15. Testing & Validation

Functional validation of the application was carried out using the sample images bundled in the uploads/ and results/ folders, covering a mix of real production-line defect captures and reference stock photography. This exercised the full upload → inference → annotation → display pipeline end to end.

15.1 Scenarios Covered

- Successful detection: fabric images containing a clear hole or yarn defect, verifying correct bounding box placement and confidence display.
- Clean sample handling: fabric images with no defect present, verifying the dashboard correctly reports zero detected objects rather than a false positive.
- Invalid file handling: the /upload endpoint rejects files outside the allowed JPG/JPEG/PNG extensions with a 400 response.
- Missing model handling: if best.pt is not found or ultralytics fails to import, the backend returns a clear 500 error instead of crashing.
- Inference timing: response payloads include measured inference_time (ms) to help characterize performance on the host hardware.

15.2 Recommended Additional Testing

- Formal evaluation against a held-out test split with a confusion matrix and per-class precision/recall breakdown.
- Stress testing the /video_feed endpoint under sustained load to characterize real-world FPS on target deployment hardware.
- Cross-browser and mobile responsiveness testing of the Bootstrap dashboard.

16. Deployment Considerations

The application currently runs via Flask's built-in development server (app.run(host='0.0.0.0', port=5000, debug=True)), which is suitable for local demonstration and testing but not recommended for production use. For a production or factory-floor deployment, the following changes are recommended:

- Serve the app behind a production WSGI server such as Gunicorn or uWSGI, fronted by Nginx.

- Disable Flask debug mode and configure proper logging and error handling.
- Containerize the application (Docker) with GPU passthrough (NVIDIA Container Toolkit) for consistent, portable deployment.
- Move uploaded/result images to persistent or cloud object storage rather than local disk, especially for multi-instance deployments.
- Add HTTPS termination and basic authentication/access control before exposing the dashboard beyond a local network.

17. Limitations & Future Work

- Recall (68.5%) indicates some defects may currently go undetected; more training data and epochs could improve this.
- Model is trained on two defect classes only (hole, yarn defect); extending to more defect types (stains, misprints, seam issues) would broaden coverage.
- Detection history and results are not persisted to a database — currently session/browser based.
- No authentication or multi-user access control is implemented.
- Live camera mode has no FPS benchmarking; real-time throughput depends on host CPU/GPU.
- Could be extended with automated pass/fail thresholds, PDF QA reports, and integration with production-line reject mechanisms.

18. Conclusion

Quality Assurance AI demonstrates a practical, end-to-end computer vision pipeline for automated textile defect inspection — combining a custom-trained YOLOv8 model with a polished, real-time web dashboard. With a precision of 81.3% and mAP@0.50 of 75.0%, the system is well-suited as a first-line automated screening tool, reducing manual inspection workload while flagging likely defects for human confirmation. The modular Flask architecture also provides a solid foundation for future expansion into additional defect classes, database-backed reporting, and production-line integration.

. Appendix A: Dependencies

- flask — Web application framework and routing.
- ultralytics — YOLOv8 model loading, training, and inference API.
- opencv-python-headless — Image/frame processing, encoding, and annotation rendering.
- werkzeug — Secure filename utilities for uploaded files.

. Appendix B: Class Index Mapping

Class ID	Class Name
0	hole
1	yarn defect